Báo cáo thực nghiệm – Pha 1

1. Thiết kế thực nghiệm

a. Môi trường

Thực nghiệm chạy trên Kaggle CPU (4 vCPU, RAM 32 GB); ứng dụng, cơ sở dữ liệu và bộ sinh tải cùng đặt trên một máy.

Thành phần	Thiết lập
Máy	Kaggle CPU: 4 vCPU (Xeon 2,2 GHz, 2 luồng/lõi, tức khoảng 2 lõi vật lý), RAM 32 GB
Ứng dụng	Heartlink (commit b7fd0cb), Java 21.0.12, Spring Boot; cấu hình mặc định (pool kết nối 10, Tomcat 200 thread theo mặc định của Spring Boot, chưa xác minh lúc chạy)
JVM	`-Xms1g -Xmx2g`, các tham số khác mặc định
Cơ sở dữ liệu	PostgreSQL 16.15, `shared_buffers=512MB`, `max_connections=200`, còn lại mặc định
Triển khai	Chạy trực tiếp, không dùng Docker (Kaggle không có Docker daemon)
Dữ liệu	50.000 người dùng giả; id được chia thành các dải riêng cho từng kịch bản để không ảnh hưởng nhau
Bộ sinh tải	Python `aiohttp`; N "người dùng ảo" gửi request liên tục, mỗi mức chạy 35 giây, bỏ 5 giây đầu để máy nóng lên
Xác thực	Token JWT tự ký bằng đúng khóa của ứng dụng, để mã hóa mật khẩu không làm nhiễu số đo
Giám sát	Mỗi giây ghi CPU (số lõi đang dùng) của ứng dụng, database và bộ sinh tải; số tiến trình database đang chờ khóa

Các chỉ số dùng trong báo cáo

- **RPS**: số request hệ thống xử lý được mỗi giây.
- **p50, p95, p99**: p95 = 95% request hoàn thành trong thời gian đó hoặc nhanh hơn, 5% còn lại chậm hơn; p50 là mức của một request điển hình, p99 phản ánh các request chậm nhất.
- **Tỉ lệ lỗi**: phần trăm request trả về lỗi hoặc quá thời gian chờ.
- **CPU**: số lõi đang dùng, hoặc mili-giây CPU tiêu tốn cho mỗi request.

b. Kịch bản thực nghiệm

Kịch bản 1: người dùng càng quẹt nhiều thì lần mở app sau có bị chậm hơn không?

1. **Tạo dữ liệu giả.** 50.000 người dùng giả trong database.
2. **Tạo tài khoản thử nghiệm với lịch sử có chủ đích.** Tạo 6 nhóm tài khoản, mỗi nhóm 16 tài khoản. Nhóm đầu chưa quẹt ai, các nhóm sau đã quẹt lần lượt 100, 1.000, 5.000, 10.000 và 20.000 hồ sơ đầu tiên (hồ sơ số 1 đến H). Mỗi tài khoản trong cùng nhóm có lịch sử giống hệt nhau.
3. **Cho 8 "người dùng ảo" mở app liên tục.** Với mỗi nhóm, một chương trình điều khiển 8 người dùng ảo cùng bấm "xem gợi ý" liên tục trong 30 giây (bỏ 5 giây đầu để máy nóng lên). Mỗi lần bấm chọn ngẫu nhiên một trong 16 tài khoản của nhóm đó. Chỉ có đọc, không ghi gì vào dữ liệu.
4. **Đo kết quả.** Với mỗi lần bấm ghi lại thời gian phản hồi; cuối mỗi nhóm tính RPS, p50, p95, p99, tỉ lệ lỗi và CPU của database. Riêng với database, chạy `EXPLAIN (ANALYZE, BUFFERS)` trên truy vấn gợi ý để lấy thời gian thực thi và số khối dữ liệu phải đọc, nhằm tách phần chậm do database khỏi phần chậm do các thành phần khác.

Lý do chọn lịch sử là các hồ sơ số 1 đến H: app luôn đưa hồ sơ nhỏ nhất mà người dùng chưa xem, nên người dùng xem lần lượt thì các hồ sơ đã quẹt luôn nằm liền nhau ở đầu danh sách.

Kịch bản 2: hệ thống chịu được bao nhiêu lượt quẹt mỗi giây?

1. **Chuẩn bị.** Dùng 5.000 tài khoản làm người quẹt và 20.000 hồ sơ (số 1 đến 20.000) làm mục tiêu.
2. **Tăng dần số người dùng ảo** lần lượt 1, 4, 16, 32 và 64; mỗi mức chạy 30 giây sau 5 giây khởi động.
3. **Mỗi lần quẹt.** Chọn ngẫu nhiên một người quẹt và một hồ sơ mục tiêu (chọn đều, chưa từng quẹt), 50% là thích và 50% là bỏ qua. Các hồ sơ mục tiêu không quẹt lại nên không tạo match: kịch bản chỉ đo đường ghi quẹt.

4. **Đo kết quả.** RPS, p50, p95, p99, tỉ lệ lỗi, CPU của ứng dụng, database và bộ sinh tải. Mức RPS ngừng tăng khi thêm người dùng ảo cho biết trần chịu tải.

Kịch bản 3: khi nhiều người cùng quẹt vào một số ít hồ sơ nổi tiếng thì hệ thống có chậm đi không?

1. **Lý do cần đo.** Dù gợi ý theo thuật toán nào, chỉ cần có một tiêu chí chung thì một số hồ sơ sẽ được đưa ra và bị quẹt nhiều hơn hẳn. Nghiên cứu trên một dịch vụ hẹn hò thật cho thấy giữa người dùng có một thứ bậc "độ hấp dẫn" rõ rệt [1], và các hệ gợi ý nói chung có xu hướng tập trung lượt hiển thị vào các mục phổ biến [2][3]. Để hai lượt quẹt cùng lúc không ghi đè lên nhau, database phải "khóa" hồ sơ của hai người liên quan cho đến khi ghi xong. Vì vậy, khi nhiều người cùng quẹt vào một hồ sơ, họ phải lần lượt chờ nhau.
2. **Dùng chung thiết lập** với kịch bản 2 (5.000 người quẹt, 32 người dùng ảo) và chạy hai lần liên tiếp.
3. **Lần 1 (đối chứng).** Chọn hồ sơ mục tiêu đều ngẫu nhiên.
4. **Lần 2 (hồ sơ nổi tiếng).** Chọn hồ sơ mục tiêu theo phân phối Zipf, thường dùng để mô phỏng độ phổ biến (như trong benchmark YCSB [5] và các nghiên cứu về truy cập web [4]). Tham số 1,2 được chọn để tạo tình huống căng, không hiệu chỉnh từ dữ liệu thật của Heartlink. Xác suất chọn hồ sơ hạng k tỉ lệ với 1/k^1,2, nên hồ sơ số 1 nhận 20,4% lượt quẹt, số 2 nhận 8,9%, số 3 nhận 5,5%, và cứ thế giảm nhanh. Theo lý thuyết, 10 hồ sơ đầu tiên nhận 50,3% tổng số lượt. Trong thực nghiệm mỗi người không quẹt lặp lại một hồ sơ, nên con số đo được là 39,4%.
5. **Đo kết quả.** Độ trễ riêng cho nhóm hồ sơ nổi tiếng (10 hồ sơ đầu) và nhóm còn lại, RPS tổng, và số tiến trình database đang chờ khóa (lấy mẫu mỗi giây).

Kịch bản 4: chat dùng polling thì tốn bao nhiêu tài nguyên?

1. **Tạo dữ liệu.** 999 cuộc trò chuyện đang hoạt động; 874 cuộc trong số đó đã có vài chục tin nhắn.
2. **Mô phỏng polling.** Mỗi người dùng ảo gọi "đọc 20 tin gần nhất" của một cuộc trò chuyện, nghỉ 2 giây rồi gọi lại, giống app chat hỏi server định kỳ. Chạy với 200 và 1.000 người dùng ảo, mỗi mức 30 giây; thời điểm bắt đầu lệch ngẫu nhiên trong 2 giây đầu để không gọi đồng loạt.
3. **Đo kết quả.** Tải lý tưởng là số người dùng ảo chia 2 giây (100 và 500 request mỗi giây). So sánh với RPS thực, độ trễ và CPU. Vì API chưa có tham số lấy tin mới hơn, mỗi lần gọi đều tải lại tối đa 20 tin dù không có tin mới.

2. Kết quả thực nghiệm

a. Kịch bản 1: người dùng càng quẹt nhiều thì lần mở app sau càng chậm

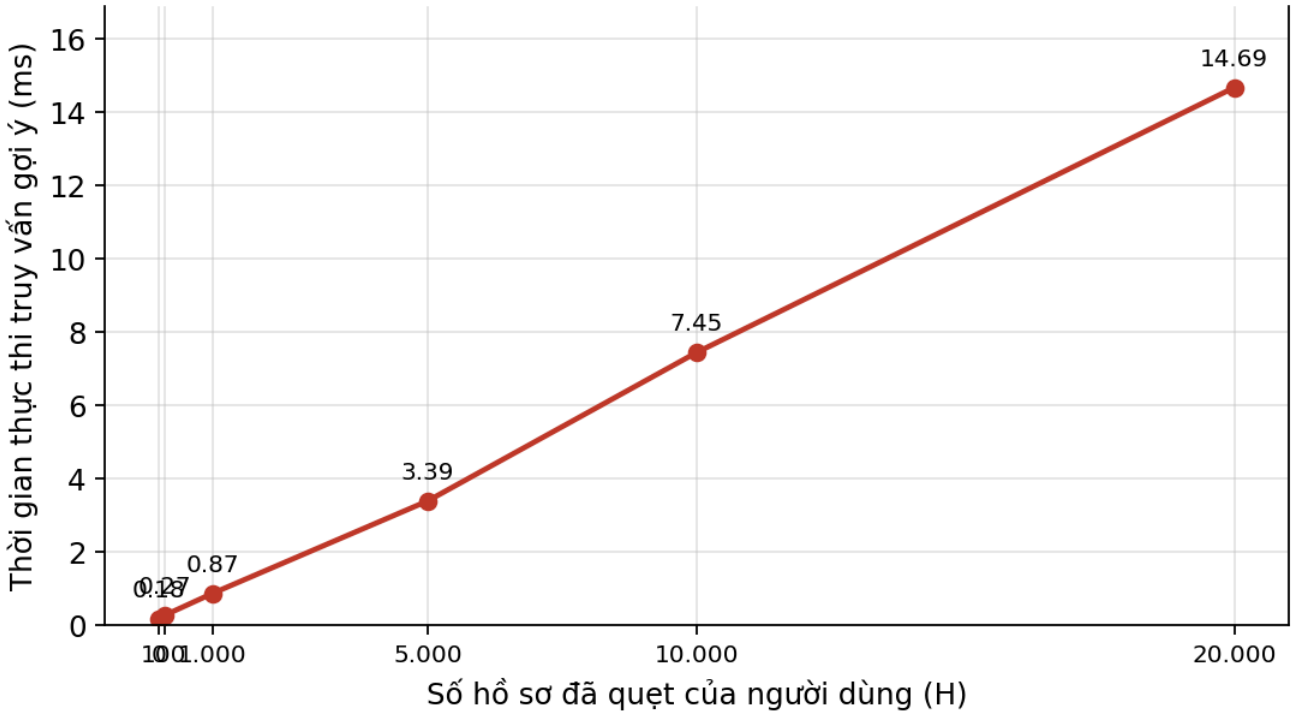
Người dùng quẹt càng nhiều thì app gợi ý càng chậm. Với người đã quẹt 20.000 hồ sơ, hệ thống chỉ xử lý được 170 lượt gợi ý mỗi giây, chậm hơn khoảng 12 lần so với người dùng mới (2.038 lượt mỗi giây).

Bảng 1. Kết quả theo số hồ sơ đã quẹt (H), 8 người dùng ảo, mỗi mức 30 giây.

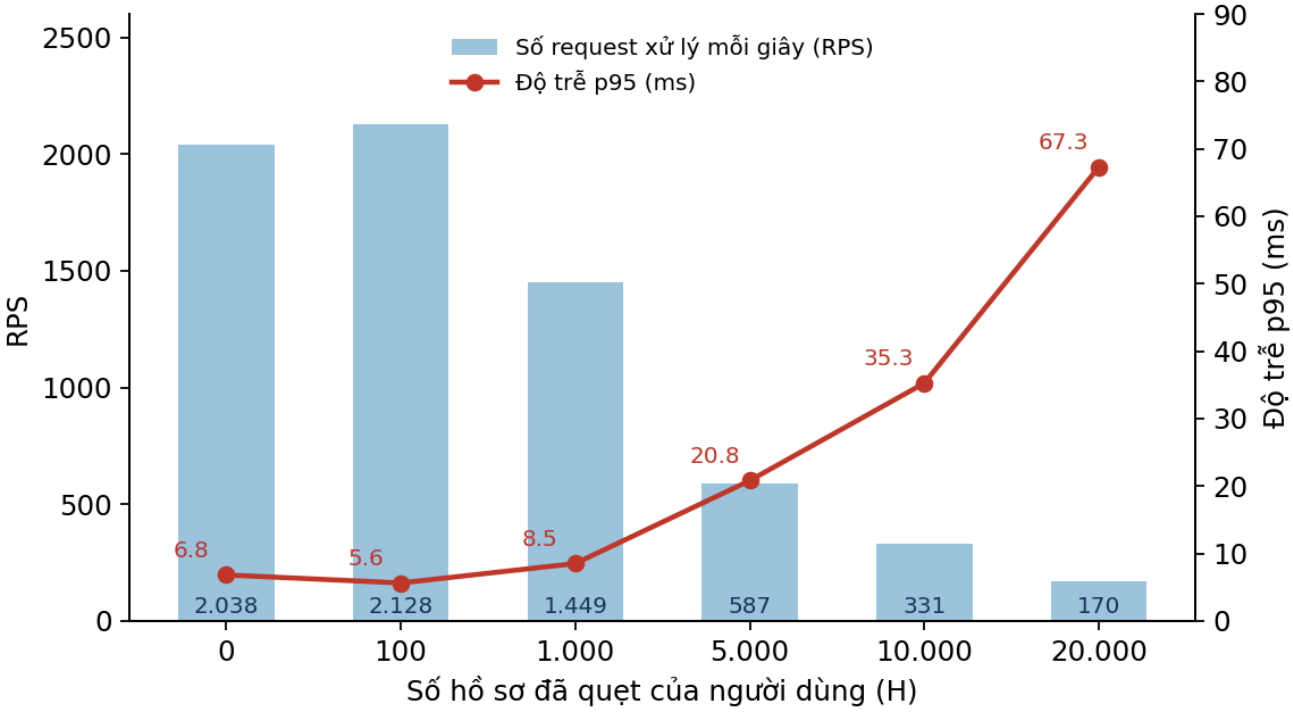
H	RPS	p50 (ms)	p95 (ms)	p99 (ms)	Thời gian SQL (ms)	Số khối đọc	CPU database mỗi request (ms)
0	2.038	3,57	6,82	9,34	0,18	9	0,60
100	2.128	3,61	5,64	7,13	0,26	12	0,68
1.000	1.449	5,32	8,52	10,31	0,87	32	1,46
5.000	587	13,29	20,82	24,58	3,39	132	5,21
10.000	331	23,97	35,26	40,59	7,45	256	10,10
20.000	170	46,56	67,29	78,50	14,69	513	20,27

Bảng 2. So sánh mức H = 20.000 với H = 0.

Chỉ số	Thay đổi
RPS	giảm khoảng 12 lần
Độ trễ p95	tăng khoảng 10 lần
Thời gian SQL	tăng khoảng 82 lần
Số khối đọc	tăng khoảng 57 lần
CPU database mỗi request	tăng khoảng 34 lần



Hình 1. Thời gian thực thi truy vấn gợi ý theo số hồ sơ đã quẹt.



Hình 2. Số request xử lý mỗi giây (cột) và độ trễ p95 (đường) theo số hồ sơ đã quẹt.

Nhận xét

- **Chậm dần đều theo số hồ sơ đã quẹt.** Thời gian chạy truy vấn gợi ý tăng từ 0,18 ms lên 14,69 ms, và tăng gần như đều đặn, không có chỗ nào đột ngột.
- **Vì sao chậm.** Mỗi lần gợi ý, hệ thống đi từ hồ sơ số 1 và phải bỏ qua tất cả hồ sơ người dùng đã quẹt rồi mới tìm đủ 20 hồ sơ chưa xem. Quẹt càng nhiều thì đoạn phải bỏ qua càng dài.
- **Người quẹt ít thì chưa thấy khác biệt.** Dưới khoảng 1.500 hồ sơ, độ chậm còn nhỏ (p95 là 8,5 ms so với 6,8 ms ở người dùng mới).

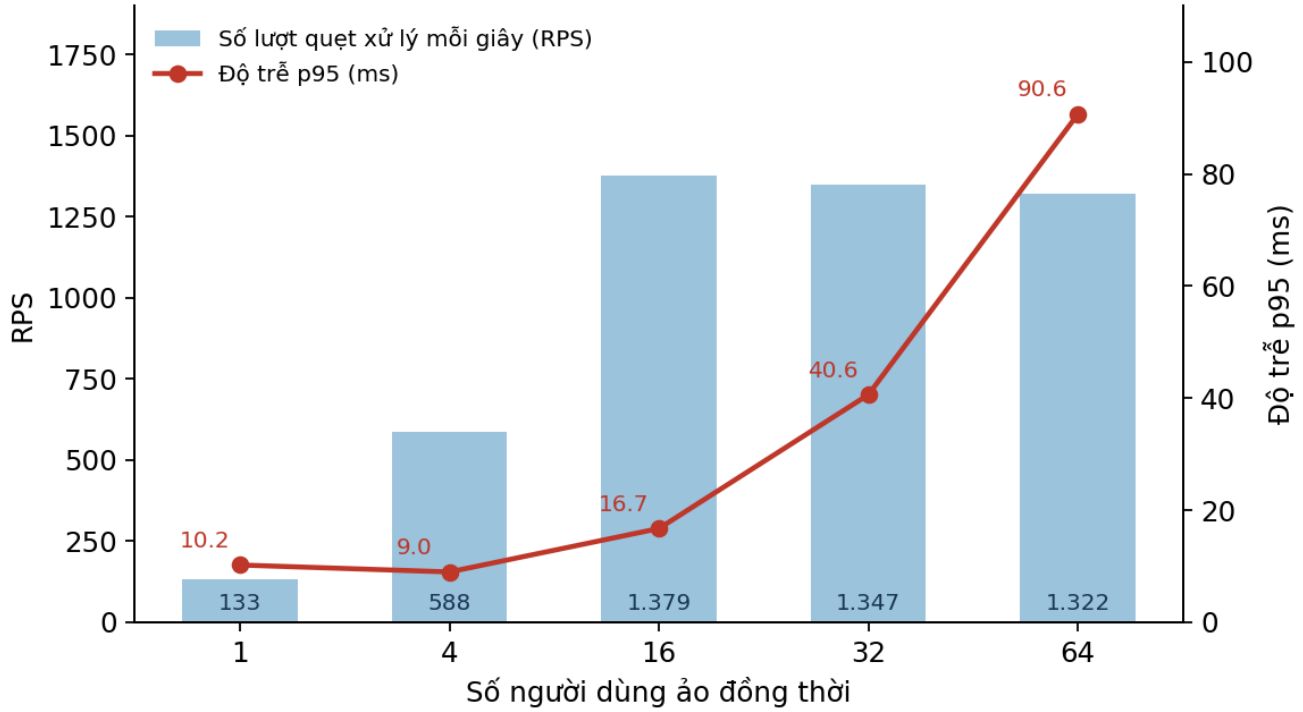
Kết luận. Chi phí gợi ý hồ sơ tăng theo lịch sử quẹt của từng người dùng. Đây là điểm cần cải thiện ở Pha 2.

b. Kịch bản 2: trần chịu tải của đường ghi quẹt

Hệ thống đạt đỉnh khoảng 1.379 lượt quẹt mỗi giây khi có 16 người dùng ảo. Thêm người nữa không làm xử lý nhanh hơn mà chỉ khiến mỗi người phải chờ lâu hơn. Không có yêu cầu nào bị lỗi.

Bảng 3. Kết quả theo số người dùng ảo đồng thời, mỗi mức 30 giây. CPU tính bằng số lõi đang dùng.

Người dùng ảo	RPS	p50 (ms)	p95 (ms)	p99 (ms)	CPU ứng dụng	CPU database	CPU bộ sinh tải
1	133	7,10	10,22	11,96	0,46	0,17	0,13
4	588	6,66	8,99	10,56	0,76	0,56	0,35
16	1.379	11,18	16,73	20,35	1,44	1,11	0,67
32	1.347	23,03	40,62	52,01	1,46	1,10	0,68
64	1.322	47,76	90,65	121,93	1,44	1,10	0,67



Hình 3. Số lượt quẹt xử lý mỗi giây (cột) và độ trễ p95 (đường) theo số người dùng ảo đồng thời.

Nhận xét

- **Tăng đến 16 người rồi đứng lại.** Từ 1 lên 16 người dùng ảo, số lượt quẹt xử lý được tăng gần như tỉ lệ thuận (133, 588, 1.379). Từ 16 trở đi nó giữ quanh 1.300 đến 1.400, còn độ trễ p95 tăng từ 16,7 ms lên 90,7 ms khi có 64 người.
- **Máy gần hết sức.** Lúc đó CPU đã dùng khoảng 3,2 trên 4 vCPU, nên đây là mức tối đa của một nút ứng dụng trên cấu hình này. Chưa rõ nút thắt là CPU hay số kết nối database mặc định (10).

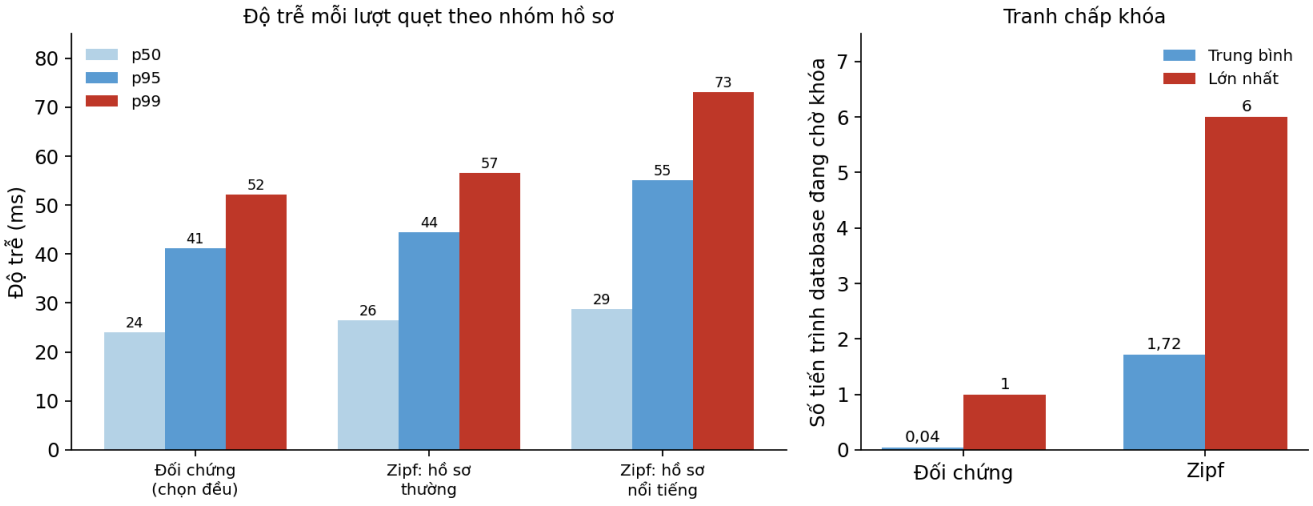
Kết luận. Một nút ứng dụng Pha 1 trên cấu hình này xử lý tối đa khoảng 1.350 đến 1.400 lượt quẹt mỗi giây. Đây là mốc để so sánh với Pha 2.

c. Kịch bản 3: hồ sơ nổi tiếng làm các lượt quẹt xếp hàng chờ nhau

Khi 39,4% lượt quẹt dồn vào 10 hồ sơ nổi tiếng, số lượt quẹt xử lý được mỗi giây giảm 14% (từ 1.299 xuống 1.119) và các hồ sơ nổi tiếng bị quẹt chậm hơn rõ rệt. Không có yêu cầu nào bị lỗi.

Bảng 4. Kết quả với 32 người dùng ảo, mỗi lần chạy 30 giây. Hồ sơ nổi tiếng là 10 hồ sơ đầu tiên.

Nhóm	Số lượt quẹt	RPS	p50 (ms)	p95 (ms)	p99 (ms)
Đối chứng (chọn đều), toàn bộ	38.964	1.299	24,06	41,16	52,08
Zipf, toàn bộ	33.564	1.119	27,17	48,96	65,58
Zipf, hồ sơ thường	20.331	678	26,40	44,45	56,51
Zipf, hồ sơ nổi tiếng	13.233	441	28,75	55,07	73,13



Hình 4. Độ trễ mỗi lượt quẹt theo nhóm hồ sơ (trái) và số tiến trình database đang chờ khóa, lấy mẫu mỗi giây (phải).

Nhận xét

- **Hồ sơ nổi tiếng chậm hơn.** So với đối chứng, p95 của nhóm này cao hơn 34% (55,07 so với 41,16 ms) và p99 cao hơn 40% (73,13 so với 52,08 ms). Nhóm hồ sơ thường cũng chậm hơn nhẹ (p95 cao hơn 8%).
- **Vì sao.** Mỗi lượt quẹt "khóa" tạm thời hồ sơ của hai người liên quan cho đến khi ghi xong, nên các lượt quẹt vào cùng một hồ sơ phải lần lượt chờ nhau. Số tiến trình database đang chờ khóa tăng từ 0,04 lên 1,72 (trung bình) và lớn nhất là 6.
- **Mức tập trung là giả định.** Con số 39,4% đến từ phân phối Zipf với tham số 1,2 chứ không đo từ người dùng thật, nên kết quả chỉ cho thấy xu hướng.

Kết luận. Hồ sơ được quẹt nhiều làm thông lượng giảm 14% và đuôi độ trễ tăng 34% đến 40%. Đây là mốc để so sánh ở Pha 2.

d. Kịch bản 4: polling chat tốn tài nguyên tỉ lệ thuận với số người dùng

Chat dùng polling nên chi phí tăng theo số người đang mở chat, kể cả khi không ai nhắn gì. Với 1.000 người mở chat và cứ 2 giây hỏi server một lần, server phải xử lý 499 yêu cầu mỗi giây và tốn khoảng 0,78 lõi CPU (khoảng 20% sức máy). Chu kỳ 2 giây là giả định của thực nghiệm.

Bảng 5. Kết quả polling, mỗi người dùng ảo gọi "đọc 20 tin gần nhất" rồi nghỉ 2 giây, mỗi mức 30 giây. CPU tính bằng số lõi đang dùng.

Người dùng ảo	Tải lý tưởng (request/giây)	RPS thực	p50 (ms)	p95 (ms)	p99 (ms)	CPU ứng dụng	CPU database	CPU bộ sinh tải
200	100	99,8	2,73	3,29	3,60	0,15	0,06	0,08
1.000	500	499,0	2,93	4,92	6,72	0,55	0,23	0,26

Nhận xét

- **Tăng đúng theo số người.** Số yêu cầu thực tế gần bằng lý thuyết (99,8 so với 100 và 499 so với 500): mỗi người mở chat tạo ra 0,5 yêu cầu mỗi giây dù có tin mới hay không.
- **Mỗi yêu cầu nhanh nhưng phần lớn là thừa.** p95 chỉ 4,9 ms, nên vấn đề không phải độ chậm mà là sự lãng phí: trong lúc đo không ai nhắn tin, nên lần nào server cũng gửi lại đúng 20 tin cũ.

Kết luận. Polling làm chi phí chat tăng theo số người mở chat, phần lớn là tải lại dữ liệu không đổi.

3. Tổng hợp painpoint và giới hạn của số liệu

Bốn thực nghiệm cho thấy Pha 1 có bốn điểm yếu định lượng được, trong đó chi phí gợi ý theo lịch sử quẹt là nghiêm trọng nhất.

Bảng 6. Tổng hợp painpoint và chỉ số dùng để so sánh với Pha 2.

Painpoint	Băng chứng chính	Mức ảnh hưởng	Chỉ số so sánh ở Pha 2
Gợi ý chậm theo lịch sử quẹt	H từ 0 lên 20.000: RPS từ 2.038 xuống 170, thời gian SQL từ 0,18 lên 14,69 ms, CPU database mỗi request tăng 34 lần	Cao	Thời gian SQL, CPU database mỗi request, RPS theo H
Trần ghi quẹt của một nút	Đỉnh 1.379 RPS ở 16 người dùng ảo; p95 từ 16,7 lên 90,7 ms khi lên 64	Vừa	RPS đỉnh, p95 ở 32 người dùng ảo
Hồ sơ nổi tiếng	Khi 39,4% lượt quẹt dồn vào 10 hồ sơ (Zipf 1,2): thông lượng giảm 14%, p99 của nhóm hồ sơ nổi tiếng tăng 40%, tối đa 6 tiến trình chờ khóa; mức lệch là giả định	Vừa	Tỉ lệ RPS giữa chia theo phân phối Zipf và phân phối đều, số tiến trình chờ khóa
Polling chat	1.000 người dùng tạo 499 request mỗi giây, tốn khoảng 0,78 lõi CPU, toàn bộ là tải lại dữ liệu không đổi	Vừa	CPU cho mỗi 1.000 người dùng, độ trễ nhận tin

Tài liệu tham khảo

- [1] E. E. Bruch, M. E. J. Newman. *Aspirational pursuit of mates in online dating markets.* Science Advances 4(8), eaap9815, 2018. https://doi.org/10.1126/sciadv.aap9815
- [2] A. Klimashevskaia, D. Jannach, M. Elahi, C. Trattner. *A survey on popularity bias in recommender systems.* 2024. https://arxiv.org/abs/2308.01118
- [3] H. Abdollahpouri, M. Mansoury, R. Burke, B. Mobasher. *The Unfairness of Popularity Bias in Recommendation.* 2019. https://arxiv.org/abs/1907.13286
- [4] L. Breslau, P. Cao, L. Fan, G. Phillips, S. Shenker. *Web Caching and Zipf-like Distributions: Evidence and Implications.* IEEE INFOCOM, 1999. https://pages.cs.wisc.edu/~cao/papers/zipf-implications.html

- [5] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, R. Sears. *Benchmarking Cloud Serving Systems with YCSB.* ACM Symposium on Cloud Computing (SoCC), 2010.